5️⃣

Overview

```
### **Genie Performance & Cost Optimization Engine**

**Problem**: Poorly optimized Genie spaces generate expensi
ve, slow SQL queries that impact warehouse costs and user e
xperience.

**Solution**: Build an accelerator that:

- **Query Pattern Analysis**: Analyzes historical Genie-gen
erated queries to identify inefficient patterns (full table
scans, missing indexes, redundant joins)
- **Automatic Materialized View Suggestions**: Recommends m
aterialized views or aggregation tables based on frequent q
uery patterns
- **Cost Prediction & Alerts**: Estimates warehouse cost be
fore executing queries and alerts when queries exceed budge
t thresholds
- **SQL Query Rewriting**: Automatically rewrites Genie-gen
erated SQL for better performance (e.g., converting subquer
ies to CTEs, optimizing joins)
- **Warehouse Right-Sizing**: Recommends optimal warehouse
sizing based on actual Genie usage patterns

**ROI Metrics**:

- 30-50% reduction in warehouse compute costs
- 60% improvement in average query response time
- $50K-$200K annual cost savings for mid-size enterprises
```

Here's the complete deep-dive on the **Genie Performance & Cost Optimization Engine** — grounded in what's actually available in Databricks today, what

GenieRx already does, and what gaps remain wide open for you to build into a killer accelerator.

🚀 Genie Performance & Cost Optimization Engine — Deep Dive

> *"Genie is only as good as the SQL it generates and the warehouse it runs on. Without optimization guardrails, you're handing business users a sports car without a speedometer."*

🎯 The Real Problem, In Precise Terms

Databricks AI/BI Genie converts natural language → SQL → results. That SQL runs against your SQL Warehouse with **zero cost awareness, zero performance guardrails, and zero feedback loop** between the generated query and your DBUs consumed. At enterprise scale:

Pain Point	Real-World Impact
Full table scans on 500M-row fact tables	10x cost spike on a single "show me last year's revenue" question
Redundant joins from poorly annotated schemas	3-5x slower queries, warehouse auto-scaling kicks in unnecessarily
Business users running the same aggregate 50 times/day	Zero caching benefit without materialized views
No warehouse sizing guidance	Teams over-provision XL warehouses for simple reporting queries

What GenieRx (github.com/hiydavid/dbx-genie-rx) already solves: It addresses Genie Space *configuration quality* — analyzing your space setup against best practices and optimizing your knowledge store instructions. That's the **supply side** (better SQL generation inputs). Your accelerator targets the **demand side** — what happens *after* Genie generates SQL and hits the warehouse.

🏗️ Architecture Blueprint

```
┌──────────────────────────────────────────────────────────────────
────────┐
│                    GENIE PERF & COST ENGINE
│
│
│
│   ┌────────────────┐     ┌────────────────┐     ┌────────────────
───┐    │
│   │  Query         │     │  MV/Agg        │     │  Cost Predicti
on  │    │
│   │  Analyzer      │────▶│  Recommender   │     │  & Alert Engin
e  │    │
│   │  (Layer 1)     │     │  (Layer 2)     │     │  (Layer 3)
│    │
│   └────────┬───────┘     └────────────────┘     └────────────────
───┘    │
│            │
│
│   ┌────────▼───────┐     ┌────────────────┐     ┌────────────────
───┐    │
│   │  SQL Rewriter│     │  Warehouse     │     │  Unified Cost
│    │
│   │  (Layer 4)     │     │  Right-Sizer   │     │  Dashboard
│    │
│   └────────────────┘     │  (Layer 5)     │     │  (Layer 6)
│    │
│                          └────────────────┘     └────────────────
───┘    │
│
│
│  DATA SOURCES: System Tables → query_history, billing, cl
usters  │
└──────────────────────────────────────────────────────────────────
────────┘
```

The **single most important architectural decision**: everything is built on **Databricks System Tables** — `system.query.history`, `system.billing.usage`,

`system.compute.clusters`. This means zero external tooling, full Unity Catalog governance.

🔬 Layer 1 — Query Pattern Analysis

What it does: Reads Genie-generated SQL from `system.query.history` (filtering by `executed_by_user_agent LIKE '%genie%'`) and classifies anti-patterns using both rule-based SQL parsing and an LLM judge.

Key Data Source:

```
-- Pull all Genie-executed queries from System Tables
SELECT
  query_id,
  query_text,
  total_duration_ms,
  execution_duration_ms,
  read_bytes,
  rows_produced,
  metrics,
  statement_type
FROM system.query.history
WHERE executed_by_user_agent LIKE '%genie%'
  AND start_time >= CURRENT_TIMESTAMP - INTERVAL 30 DAYS
ORDER BY total_duration_ms DESC;
```

Anti-Pattern Detection Rules:

Pattern	Detection Logic	Severity
Full table scan	`read_bytes / rows_produced > threshold` + no Z-order/partition filter in SQL	🔴 Critical
Cross-join explosion	SQL contains `CROSS JOIN` or cartesian product without `WHERE` bounds	🔴 Critical
Repeated subquery	Same `SELECT` block appearing 3+ times in query text	🟡 Warning
Missing aggregate cache	Same aggregation query run >10x in 24hrs with identical results	🟡 Warning

Pattern	Detection Logic	Severity
Anti-Delta scan (no predicate pushdown)	Query on partitioned table without partition column in `WHERE`	🟡 Warning
Redundant sort	`ORDER BY` on final result with no `LIMIT` clause	🟢 Info

LLM-Powered Classification (using Databricks-hosted Claude Sonnet):

```
# Each query gets scored by an LLM judge for semantic anti-
patterns
prompt = f"""
Analyze this Databricks SQL query generated by AI/BI Genie.
Identify: 1) Inefficient JOIN ordering, 2) Subqueries repla
ceable with CTEs,
3) Missing WHERE clause selectivity, 4) Aggregation that sh
ould be pre-computed.

Query: {query_text}
Table Stats: {table_stats_json}

Return JSON with: [pattern_type, severity, line_number, sug
gested_fix]
"""
```

📐 Layer 2 — Automatic Materialized View Suggestions

The insight: If the same aggregation pattern (e.g., `GROUP BY customer_id, month, region`) appears in Genie-generated queries more than N times per week, a materialized view will eliminate 90%+ of that compute.

Clustering Algorithm:

```
from databricks.sdk import WorkspaceClient
import sqlglot
from collections import Counter

def extract_query_signatures(query_text: str) -> dict:
```

```
    """Normalize SQL to extract structural fingerprint"""
    tree = sqlglot.parse_one(query_text, dialect="spark")
    return {
        "tables": sorted([t.name for t in tree.find_all(sql
glot.exp.Table)]),
        "group_by_cols": [c.name for c in tree.find_all(sql
glot.exp.Group)],
        "aggregations": [a.sql() for a in tree.find_all(sql
glot.exp.Anonymous)],
        "time_grain": detect_time_grain(tree)  # DAY/WEEK/M
ONTH
    }

# Cluster similar signatures → recommend one MV per cluster
signature_counts = Counter([
    json.dumps(extract_query_signatures(q), sort_keys=True)
    for q in genie_query_history
])
```

MV Recommendation Output:

```
-- RECOMMENDED: Create this Materialized View to serve 47 G
enie queries/week
-- Estimated DBU savings: 312 DBUs/month (~$468 at $1.50/DB
U)

CREATE MATERIALIZED VIEW gold.mv_sales_by_customer_month
SCHEDULE CRON '0 2 * * *'  -- Refresh nightly
AS
SELECT
  customer_id,
  DATE_TRUNC('month', order_date) AS order_month,
  region,
  SUM(revenue) AS total_revenue,
  COUNT(DISTINCT order_id) AS order_count
FROM silver.orders
GROUP BY 1, 2, 3;
```

Important 2025 Context: Databricks now supports **MV refresh tracking in** `DESCRIBE EXTENDED AS JSON` and **Unity Catalog Metric Views** (GA May 2025) which are natively recognized by Genie — making your MV recommendations directly consumable by Genie without additional configuration. Source

---

💰 Layer 3 — Cost Prediction & Alerts

The mechanism: Before Genie executes a query, the engine intercepts it via a **Databricks App** proxy layer, runs a cost estimation model, and either allows/blocks/warns.

Cost Estimation Model (trained on your own `system.billing.usage`):

```
# Feature engineering from query characteristics
features = {
    "estimated_bytes_scanned": get_table_size_estimate(tabl
es, filters),
    "join_complexity_score": count_joins(query_ast) * join_
penalty,
    "aggregation_cardinality": estimate_group_by_cardinalit
y(group_cols),
    "warehouse_size_dbu_rate": warehouse_config["dbu_per_ho
ur"],
    "historical_similar_query_cost": lookup_similar_queries
(query_sig)
}

# XGBoost regression model trained on (query_features → act
ual_cost)
predicted_dbu = cost_model.predict(features)
predicted_usd = predicted_dbu * workspace_dbu_price
```

Alert Tiers:

Threshold	Action
< $1	✅ Execute silently
$1 - $10	⚠️ Soft warning shown to user with estimated cost
$10 - $50	🔴 Require user confirmation with optimization suggestions

Threshold	Action
> $50	🚫 Block + auto-create rewrite suggestion + Slack alert to data team

Budget Tracking Dashboard (built as a Databricks App):

- Per-Genie-Space daily/weekly/monthly spend
- Per-user cost attribution
- Anomaly detection: "This Genie Space's cost is 340% above 30-day average"

✍️ Layer 4 — SQL Query Rewriting

This is where the LLM-powered engine earns its ROI. The rewriter takes Genie's raw SQL and produces an optimized version using Databricks-specific constructs.

Rewrite Rules Catalog:

Rule 1: Subquery → CTE Conversion

```
-- BEFORE (Genie-generated)
SELECT customer_id, total_orders
FROM (
  SELECT customer_id, COUNT(*) as total_orders
  FROM orders GROUP BY customer_id
) t
WHERE total_orders > 10;

-- AFTER (Rewritten)
WITH customer_orders AS (
  SELECT customer_id, COUNT(*) as total_orders
  FROM orders GROUP BY customer_id
)
SELECT customer_id, total_orders
FROM customer_orders
WHERE total_orders > 10;
-- ✅ Photon engine optimizes CTEs more effectively than su
bqueries
```

Rule 2: Inject Delta Partition Pruning

```
-- BEFORE: Missing partition filter
SELECT * FROM silver.events WHERE event_type = 'purchase';

-- AFTER: Partition predicate injected (if table is Z-order
ed/partitioned by date)
SELECT * FROM silver.events
WHERE event_date >= CURRENT_DATE - INTERVAL 90 DAYS  -- inj
ected
  AND event_type = 'purchase';
-- ✅ Reads 90% fewer files
```

Rule 3: APPROX functions for exploratory queries

```
-- BEFORE: Exact COUNT DISTINCT (very expensive at scale)
SELECT COUNT(DISTINCT user_id) FROM events;

-- AFTER: Approximate with accuracy label
SELECT APPROX_COUNT_DISTINCT(user_id) AS approx_unique_user
s  -- ±2% error
FROM events;
-- ✅ 10-100x faster for analytical exploration
```

Rule 4: Bloom Filter Awareness

```
# Check if columns used in JOINs have bloom filters
def suggest_bloom_filter(table: str, join_column: str) -> s
tr:
    stats = get_delta_stats(table)
    if join_column not in stats.get("bloom_filter_columns",
[]):
        return f"ALTER TABLE {table} SET TBLPROPERTIES ('de
lta.dataSkippingNumIndexedCols' = 32);"
```

The rewrite workflow:

```
Genie SQL → Rule Engine (fast, deterministic) → LLM Rewrite
r (semantic)
```

```
          → Explain Plan Comparison → A/B cost estimate → Pr
esent to user
```

⚙️ Layer 5 — Warehouse Right-Sizing

Problem: Teams default to `Large` or `X-Large` warehouses out of fear of slowness, wasting 60-70% of compute capacity for Genie's typical query profile.

Sizing Engine Logic:

```
def recommend_warehouse_size(genie_space_id: str, lookback_
days: int = 30) -> dict:
    """
    Analyzes actual Genie query concurrency and P90 duratio
n
    to recommend optimal warehouse configuration
    """
    query_stats = get_genie_query_stats(genie_space_id, loo
kback_days)

    peak_concurrency = query_stats["p95_concurrent_querie
s"]
    p90_duration_ms = query_stats["p90_execution_duration_m
s"]
    avg_bytes_scanned = query_stats["avg_bytes_scanned"]

    # Right-size formula
    if peak_concurrency <= 3 and avg_bytes_scanned < 10_GB:
        recommendation = "Small (2X-Small to Small) + Auto-
Stop 10min"
    elif peak_concurrency <= 10 and p90_duration_ms < 5000:
        recommendation = "Medium with Serverless (auto-scal
ing handles peaks)"
    else:
        recommendation = "Large + Serverless fallback for b
urst"
```

```
    # Serverless vs Classic analysis
    serverless_cost = estimate_serverless_cost(query_stats)
    classic_cost = estimate_classic_cost(query_stats, recom
mendation)

    return {
        "recommended_size": recommendation,
        "serverless_saves": classic_cost - serverless_cost,
        "auto_stop_recommendation": calculate_optimal_autos
top(query_stats),
        "cluster_policy_yaml": generate_cluster_policy(reco
mmendation)
    }
```

Key 2025 Databricks Feature Leverage:

- **Serverless SQL Warehouses** now have **Predictive Query Execution (PQE)** — dynamically adjusts mid-execution to avoid skew. Your engine should flag workloads that benefit from Serverless migration.
- **Default Warehouse Setting** (Beta, Aug 2025) — your engine can programmatically set the optimal warehouse as the workspace default for Genie. Source

---

📊 Unified Cost Dashboard

Built as a **Databricks App** with AI/BI Dashboard underneath:

```
┌──────────────────────────────────────────────────────────────┐
│  GENIE COST INTELLIGENCE HUB                                 │
│                                                              │
│  This Month: $4,230    vs Budget: $5,000   ✅ On Track       │
│  MV Savings Realized:  $1,890              🎯 47%           │
│  Queries Rewritten:    234                 ⚡ 61% faster     │
│                                                              │
│  TOP 5 EXPENSIVE GENIE QUERIES THIS WEEK                     │
│  1. "Show revenue by region last 3 years" - $28.40          │
│  2. "List all customer orders" - $19.70                     │
│  3. "Forecast next quarter sales" - $15.20                  │
│                                                              │
```

```
│  RECOMMENDATIONS (3 actions, save $1,200/month)          │
│  ✦ Create MV: mv_revenue_by_region_month                 │
│  ✦ Resize Space A warehouse: XL → M (Serverless)         │
│  ✦ Add partition filter to Space B query template        │
└──────────────────────────────────────────────────────────┘
```

🛠️ Tech Stack

Component	Technology	Why
Query History Ingestion	`system.query.history` System Tables	Native, governed, no extra pipeline
SQL Parsing	`sqlglot` (Python)	Dialect-aware, Spark SQL support
Pattern Clustering	Databricks ML (MLflow + sklearn)	Keeps models in Databricks
LLM Rewriting	Claude Sonnet via Databricks Model Serving	Same as GenieRx pattern
Cost Model	XGBoost via MLflow	Lightweight, explainable
UI/Dashboard	Databricks App + AI/BI Dashboard	Native, no external hosting
Alerts	Databricks SQL Alerts (GA Nov 2025)	Native threshold alerts
Configuration	Unity Catalog Metric Views	Feeds back into Genie spaces

---

🔗 How It Complements GenieRx

Capability	GenieRx	Your Engine
Space config analysis	✅ Core feature	❌ Out of scope
Benchmark scoring	✅ Core feature	❌ Out of scope
Query pattern analysis	❌	✅ Core feature
Materialized view suggestions	❌	✅ Core feature
Cost prediction/alerts	❌	✅ Core feature
SQL auto-rewriting	❌	✅ Core feature
Warehouse right-sizing	❌	✅ Core feature

These two tools are genuinely complementary. GenieRx optimizes *what goes into Genie*, yours optimizes *what comes out and what it costs*. Together they form a complete Genie optimization loop — a strong pitch for co-publishing or cross-referencing in the Databricks Industry Solutions ecosystem.

📈 Realistic ROI Model

Optimization Lever	Mechanism	Expected Savings
Materialized Views	Eliminate repeated aggregations	25-35% compute reduction
Query Rewriting	Partition pruning + CTE + APPROX	15-25% duration improvement
Warehouse Right-Sizing	Match size to actual concurrency	20-30% warehouse spend
Cost Alerts	Block runaway queries	Prevent 5-10% cost spikes
Combined	Compounding effect	**40-55% total cost reduction**

For a mid-size enterprise running **$200K/year on Genie-adjacent SQL Warehouse compute**, this translates to **$80K-$110K annual savings** — very achievable and fully auditable through `system.billing.usage`.

🗺️ Build Roadmap

Phase 1 (4 weeks): Query analyzer + system tables ingestion pipeline + basic cost dashboard

Phase 2 (4 weeks): MV recommendation engine + SQL rewriter (rule-based first, LLM second)

Phase 3 (3 weeks): Cost prediction model + alert engine + warehouse right-sizer

Phase 4 (2 weeks): Databricks App packaging + GenieRx integration + documentation

Reference repos to build on:

- databrickslabs/dqx — for data quality patterns in your pipeline
- databrickslabs/blueprint — project scaffolding template

- hiydavid/dbx-genie-rx — architecture pattern to follow for Databricks App deployment

Want me to go deeper on any specific layer — for example, the full SQL rewriting engine logic, the cost estimation model training approach, or the Databricks App architecture for the proxy layer?

But in Genie it will create the SQL and run that automatically. How Layer 4 — SQL Query Rewriting can be developed?

Excellent question, Sourav — and this is the **most architecturally critical** thing to get right. Let me be completely straight with you.

🎯 The Hard Truth: Layer 4 Cannot Be a Runtime Interceptor

Let's cut straight to it. **Genie's SQL generation and execution is a black box.** There is no middleware hook, no pre-execution interceptor, no event that fires between "SQL generated" and "SQL runs." The platform does not expose that control point. End of story.

Here's what actually happens inside Genie:

```
User types question
      ↓
Genie LLM generates SQL  [← you have NO access here]
      ↓
SQL auto-executes on warehouse  [← you have NO access here
either]
      ↓
Results returned to user
```

So the original Layer 4 design as a **runtime SQL interceptor is not achievable** through native Genie UI. But here's where it gets interesting — there are **three legitimate architectural patterns** that absolutely work, each with different trade-offs.

✅ The Three Real Approaches to SQL Rewriting

🏆 Approach 1 — The "Upstream Injection" Pattern *(Recommended)*

> *"If you can't intercept after generation, control what gets generated in the first place"*

Core Insight: Genie generates SQL based on **Example SQL Queries** in its Knowledge Store. If you inject pre-optimized, pre-rewritten SQL templates as example queries via the **Management API**, you are effectively rewriting SQL before it ever gets generated badly.

This is the **only true pre-execution influence** you have over Genie SQL.

```
┌──────────────────────────────────────────────────────────┐
│           UPSTREAM INJECTION FLOW                        │
│                                                          │
│  system.query.history (post-execution analysis)          │
│          ↓                                               │
│  Detect bad SQL patterns from past queries               │
│          ↓                                               │
│  LLM rewrites bad SQL → optimized version                │
│          ↓                                               │
│  PATCH /api/2.0/genie/spaces/{id}                       │
│  (inject optimized SQL as new example_question_sql)      │
│          ↓                                               │
│  Next time user asks similar question...                 │
│  Genie uses YOUR optimized SQL as the template 🎯        │
└──────────────────────────────────────────────────────────┘
```

Full Implementation:

```
from databricks.sdk import WorkspaceClient
import sqlglot
import json

w = WorkspaceClient()

def analyze_and_inject_optimized_queries(space_id: str):
    """
    Core rewrite engine: analyze bad queries from history,
```

```
    rewrite them, inject back as Genie example SQL
    """

    # Step 1: Pull Genie-generated bad queries from system
tables
    bad_queries = spark.sql("""
        SELECT
            query_text,
            total_duration_ms,
            read_bytes,
            metrics
        FROM system.query.history
        WHERE executed_by_user_agent LIKE '%genie%'
          AND start_time >= CURRENT_TIMESTAMP - INTERVAL 7
DAYS
          AND total_duration_ms > 10000  -- queries > 10 se
conds
          AND read_bytes > 10737418240   -- scanned > 10 GB
        ORDER BY total_duration_ms DESC
        LIMIT 20
    """).collect()

    # Step 2: Rewrite each bad query using LLM
    optimized_examples = []
    for row in bad_queries:
        rewritten_sql = rewrite_with_llm(row.query_text)

        # Step 3: Map query back to likely user question
        question = infer_question_from_sql(row.query_text)

        optimized_examples.append({
            "id": generate_hex_id(),  # 32-char hex
            "question": [question],
            "sql": [rewritten_sql],
            "usage_guidance": [
                f"Optimized version. Original query scanned
"
                f"{row.read_bytes/1e9:.1f}GB in {row.total_
```

```
duration_ms/1000:.1f}s"
            ]
        })

    # Step 4: Fetch current space config
    space = w.genie.get_space(
        space_id=space_id,
        include_serialized_space=True
    )
    space_config = json.loads(space.serialized_space)

    # Step 5: Merge new optimized examples into existing on
es
    existing_examples = space_config["instructions"].get(
        "example_question_sqls", []
    )

    # Add new, deduplicate by question similarity
    merged = deduplicate_by_question(existing_examples + op
timized_examples)
    space_config["instructions"]["example_question_sqls"] =
sorted(
        merged, key=lambda x: x["id"]  # API requires sorte
d by id
    )

    # Step 6: Push back to Genie Space via Management API
    w.genie.update_space(
        space_id=space_id,
        serialized_space=json.dumps(space_config)
    )

    print(f"✅ Injected {len(optimized_examples)} optimized
queries into space")
```

The Rewrite LLM Function:

```
def rewrite_with_llm(original_sql: str) -> str:
    """
    Uses Databricks-hosted Claude to rewrite SQL
    with Databricks-specific optimizations
    """
    import mlflow.deployments
    client = mlflow.deployments.get_deploy_client("databric
ks")

    prompt = f"""
    You are a Databricks SQL optimization expert.
    Rewrite the following SQL query to be more performant o
n Databricks/Delta Lake.

    Apply these optimizations where applicable:
    1. Convert nested subqueries to CTEs
    2. Add partition filter hints if table uses date-based
partitioning
    3. Replace COUNT(DISTINCT x) with APPROX_COUNT_DISTINCT
(x)
       for exploratory queries (add comment noting ~2% erro
r)
    4. Convert correlated subqueries to JOINs
    5. Push filters as early as possible (predicate pushdow
n)
    6. Use QUALIFY instead of subquery for window function
filtering
    7. Leverage Liquid Clustering by including clustered co
lumns in WHERE

    ORIGINAL SQL:
    {original_sql}

    Return ONLY the optimized SQL. No explanation.
    Must be valid Databricks SQL syntax.
    """

    response = client.predict(
```

```
        endpoint="databricks-claude-sonnet-4-5",
        inputs={"messages": [{"role": "user", "content": pr
ompt}]}
    )

    return response["choices"][0]["message"]["content"]
```

Why this works: Genie learns from example SQL. When a user asks a question matching your optimized template, Genie uses **your rewritten SQL** instead of generating a new inefficient one. The Trusted Asset mechanism even marks it as verified.

🥈 Approach 2 — The "Genie API Wrapper" Pattern *(For Custom App Deployments)*

If you're building a **custom chat interface** (Databricks App, Slack bot, Teams integration) that calls Genie via the Conversation API — then you **do** have a natural intercept point. The flow becomes:

```
User's question
      ↓
Your Databricks App / Wrapper Layer  ← YOU OWN THIS
      ↓
POST /api/2.0/genie/spaces/{id}/start-conversation
      ↓
Poll GET message until status = COMPLETED
      ↓
Extract generated SQL from attachments[].query.statement ←
INTERCEPT HERE
      ↓
Run SQL Rewriter on extracted SQL
      ↓
Execute REWRITTEN SQL via Statement Execution API  ← YOU RU
N THIS
      ↓
Return results to user (bypassing Genie's execution)
```

Full Code Pattern:

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.sql import StatementState
import time

w = WorkspaceClient()

def genie_with_sql_rewrite(
    space_id: str,
    warehouse_id: str,
    user_question: str
) -> dict:
    """
    Custom Genie wrapper with SQL rewrite before execution
    """

    # Step 1: Ask Genie to generate SQL (don't care about i
ts execution)
    conv = w.genie.start_conversation_and_wait(
        space_id=space_id,
        content=user_question
    )

    # Step 2: Extract the generated SQL from attachments
    generated_sql = None
    attachment_id = None

    for attachment in (conv.attachments or []):
        if hasattr(attachment, 'query') and attachment.quer
y:
            generated_sql = attachment.query.query  # The S
QL string
            attachment_id = attachment.attachment_id
            break

    if not generated_sql:
        # Genie answered with text, no SQL needed
        return {"type": "text", "content": conv.content}
```

```
    # Step 3: ⚡ REWRITE THE SQL BEFORE EXECUTION
    rewritten_sql, optimization_notes = rewrite_sql_pipelin
e(generated_sql)

    # Step 4: Execute YOUR rewritten SQL via Statement Exec
ution API
    # (not Genie's auto-execution)
    stmt = w.statement_execution.execute_statement(
        warehouse_id=warehouse_id,
        statement=rewritten_sql,
        wait_timeout="30s"
    )

    # Step 5: Return results with transparency
    return {
        "type": "query_result",
        "original_sql": generated_sql,
        "optimized_sql": rewritten_sql,
        "optimizations_applied": optimization_notes,
        "results": stmt.result,
        "performance_delta": calculate_savings(generated_sq
l, rewritten_sql)
    }
```

The Multi-Stage Rewrite Pipeline:

```
def rewrite_sql_pipeline(sql: str) -> tuple[str, list]:
    """
    Stage 1: Rule-based fast rewrites (deterministic)
    Stage 2: LLM semantic rewrites (intelligent)
    Stage 3: Validation (ensure rewrite is equivalent)
    """
    notes = []
    current_sql = sql

    # ── Stage 1: Rule-Based Rewrites ──────────────────

    # Rule: Add LIMIT if missing on large table queries
```

```
    if not has_limit(current_sql) and estimates_large_scan
(current_sql):
        current_sql = add_safety_limit(current_sql, limit=1
0000)
        notes.append("Added LIMIT 10000 safety guard (no LI
MIT detected on large table)")

    # Rule: Convert SELECT * to explicit columns if table >
50 cols
    if "SELECT *" in current_sql.upper():
        wide_tables = detect_wide_tables(current_sql)
        if wide_tables:
            current_sql = expand_select_star(current_sql, w
ide_tables)
            notes.append(f"Expanded SELECT * on {wide_table
s} (reduces data transfer)")

    # Rule: Add date partition filter if missing
    partition_hint = inject_partition_filter(current_sql)
    if partition_hint != current_sql:
        current_sql = partition_hint
        notes.append("Injected partition filter for Delta t
able file pruning")

    # Rule: Replace COUNT(DISTINCT) with APPROX for explora
tory
    if is_exploratory_query(current_sql) and "COUNT(DISTINC
T" in current_sql.upper():
        current_sql = replace_exact_with_approx(current_sq
l)
        notes.append("Replaced COUNT(DISTINCT) with APPROX_
COUNT_DISTINCT (~2% error, 10x faster)")

    # ── Stage 2: LLM Rewrite (only if rules found issues)
──
    if needs_structural_rewrite(current_sql):
        llm_rewrite = rewrite_with_llm(current_sql)
```

```
        # Validate LLM rewrite is syntactically valid
        if validate_sql_syntax(llm_rewrite):
            current_sql = llm_rewrite
            notes.append("LLM structural optimization: subq
ueries→CTEs, JOIN ordering")
        else:
            notes.append("LLM rewrite rejected (invalid syn
tax), rule-based only")

    # ── Stage 3: Semantic Equivalence Check ──────────────
    # Compare EXPLAIN plans — if dramatically different, fl
ag for review
    if not plans_are_equivalent(sql, current_sql):
        notes.append("⚠ Rewrite changes query semantics — s
howing original")
        return sql, ["Rewrite rejected: semantic equivalenc
e check failed"]

    return current_sql, notes
```

Critical caveat for Approach 2: This only works when you build your **own custom Genie-powered interface**. If users are hitting the native Genie UI directly in the Databricks workspace, you cannot intercept. This is the trade-off — control vs. convenience.

🥉 Approach 3 — The "Post-Execution Learning Loop" *(Always applicable)*

Even if you can't rewrite in real-time, you can build a **continuous learning engine** that closes the optimization loop over time. This always works, regardless of how users access Genie.

```
┌──────────────────────────────────────────────────────────────┐
│          POST-EXECUTION LEARNING LOOP                        │
│                                                              │
│  Every night (DLT Pipeline / Workflow):                      │
│                                                              │
│  1. Pull Genie queries from system.query.history             │
│  2. Classify: GOOD (< 5s, < 1GB) vs BAD (else)              │
```

```
│  3. For each BAD query:                                    │
│     a. LLM rewrites it                                     │
│     b. EXPLAIN plan comparison → validate improvement      │
│     c. Cost estimate: before vs after                      │
│  4. Auto-inject top 5 rewrites as example_question_        │
│     sqls via Management API (PATCH space config)           │
│  5. Next day: measure if Genie uses new template           │
│     (check if query_text now matches injected SQL)         │
│  6. Score improvement → feedback into rewrite model        │
└────────────────────────────────────────────────────────────┘
```

This is less dramatic than real-time interception but is **fully automated, zero user disruption, and compounds over time.** After 30 days of running, your Genie space's SQL generation quality improves dramatically because all the bad patterns have been replaced with optimized templates.

🗺️ Decision Matrix: Which Approach For You?

Scenario	Best Approach
Users access Genie directly via Databricks UI	**Approach 1** (Upstream Injection only option)
Building a custom chatbot/app on top of Genie API	**Approach 2** (true runtime rewrite)
Want continuous improvement with zero disruption	**Approach 3** (post-execution learning)
Production accelerator (recommended combo)	**Approach 1 + 3 together**

🔄 The Recommended Combined Architecture

```
┌──────────────────────────────────────────────────────────────
```

```
│
│   ┌──────────────
```

```
│
│   ┌──────────────────────────────────────────────────────────────
──┐    │
│   │ Track: Did Genie use injected SQL vs. generate new?
│    │
│   │ Measure: Before/After DBU cost per query pattern
│    │
│   │ Alert: If new bad patterns emerge
│    │
│   └──────────────────────────────────────────────────────────────
──┘    │
└──────────────────────────────────────────────────────────────
──────┘
```

⚠️ The Honest Constraint Summary

What You CAN Do	What You CANNOT Do
✅ Inject optimized SQL as example queries (Management API)	❌ Intercept SQL between generation and execution in native Genie UI
✅ Build a custom wrapper app with true runtime interception	❌ Modify Genie's internal LLM prompt directly
✅ Analyze generated SQL post-execution via system tables	❌ Prevent a bad query from running the first time it's generated
✅ Auto-update Genie space config programmatically	❌ Hook into native Genie UI event pipeline
✅ Use Trusted Assets (parameterized SQLs) to guarantee pattern use	❌ Force Genie to always use your SQL for free-text questions

The bottom line: **Layer 4 is absolutely buildable — but its implementation is async and upstream, not real-time runtime.** The Approach 1 + 3 combination achieves the same cost reduction goal through a continuous improvement loop rather than a live intercept. For custom application deployments using the Genie Conversation API, Approach 2 gives you the true runtime rewrite capability.

Want me to go deeper on the Approach 2 custom wrapper architecture, or the automated nightly injection pipeline from Approach 1+3?